Interface

11 March 2024 23:20

What is an Interface?

An interface in Java is blueprint of a class. It has static constants and abstract methods.

An interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java Interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

Java interface also represents IS-A relationship.

It cannot be instantiated just like abstract class.

Why use interface?

There are mainly three reason to use interface. They are given below.

- It is used to achieve abstraction
- By interface, we can the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

How to create interface?

Syntax:

```
public interface <interface_name>{
    //declare constant fields
    //declare methods that are abstract by default.
}
```

```
public interface Printable
{
   int MAX = 100;
   void myPrint(String str);
}
```

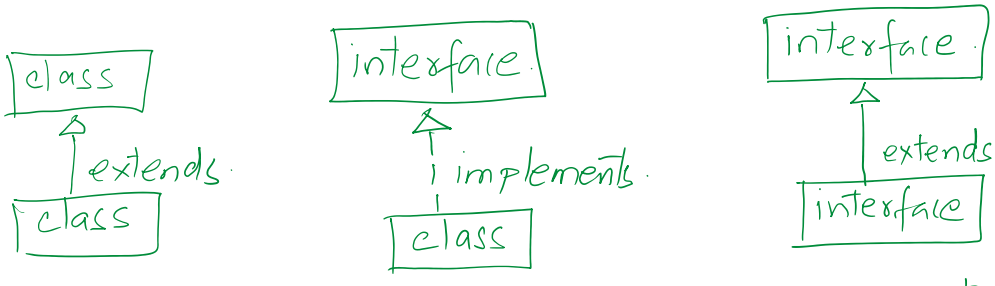
Printable.java.

compiler

```
public interface Printable
{
   public static final int MAX = 100;
   public abstract void myPrint(String str);
}
```

Printable.class.

The relationship between classes and interfaces

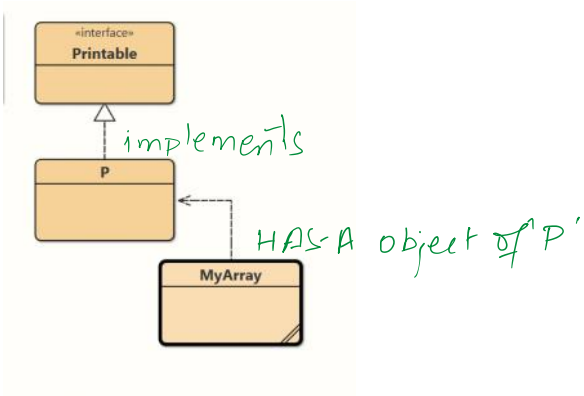


As shown in above figure, a class extends another class, an interface extends another interface, but a class implements an interface.

```
public class P implements Printable
{
}
```

InterfacesInJava.P is not abstract and does not override abstract method myPrint(java.lang.String) in InterfacesInJava.Printable

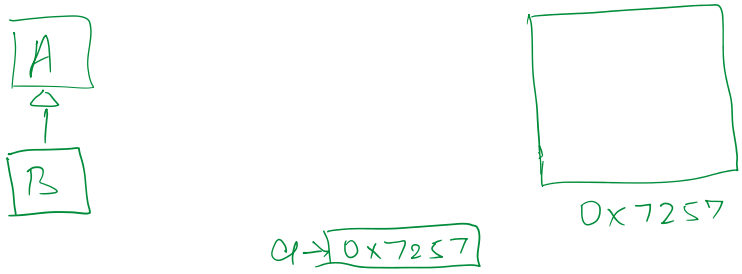
implement this method in our class



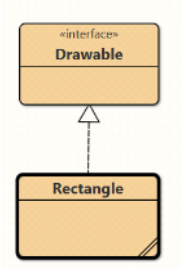

```
public interface Printable
{
   /**
    * An example of a method header - replace this comment with your own
    *
    * @param  y a sample parameter for a method
    * @return   the result produced by sampleMethod
    */
   int MAX = 100;
   void myPrint(String str);
}

public class P implements Printable
{
   public void myPrint(String str){
      System.out.print(str);
   }
}
import java.util.*;


/**
 * Write a description of class MyArray here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyArray
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      P p = new P();
      p.myPrint("\fEnter the size of the array: ");
      int i, size = sc.nextInt();
      int A[] = new int[size];
      for(i = 0; i < size; i++){
         p.myPrint("Enter the value in A["+i+"]: ");
         A[i] = sc.nextInt();
      }
      p.myPrint("\nThe array A[]: ");
      p.myPrint(Arrays.toString(A));
   }
}
```

reference variable. which is keeping the reference of child class object.

A a = new B();



From Java 8 version Default Method in Interface was introduced



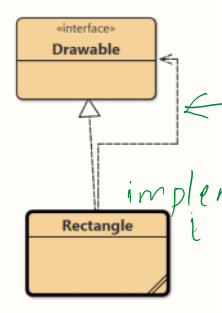

```
public interface Drawable
{
   /**
    * An example of a method header - replace this comment with your own
    *
    * @param  y a sample parameter for a method
    * @return   the result produced by sampleMethod
    */
```

method of interface

```
    * @param  y a sample parameter for a method
    * @return   the result produced by sampleMethod
    */
   default void message(){
      System.out.print("\nDefault method.");
   }
}

public class Rectangle implements Drawable
{
   public void drawRectangle(){
      System.out.print("\nDrawing Rectangle.\n");
   }

   public static void main(String args[]){
      Rectangle r1 = new Rectangle();
      r1.drawRectangle();
      r1.message(); //calling default method of an interface.
   }
}
```

Default method of interface

Java 8 static method in interface



we are calling static method of Drawable class, by using Drawable as reference.

implements Drawable

```
public interface Drawable
{
   /**
    * An example of a method header - replace this comment with your own
    *
    * @param  y a sample parameter for a method
    * @return   the result produced by sampleMethod
    */
   default void message(){
      System.out.print("\nDefault method.");
   }

   static double area(double length, double breadth){
      return length * breadth;
   }

   static double perimeter(double length, double breadth){
      return 2*(length + breadth);
   }
}
public class Rectangle implements Drawable
{
   public void drawRectangle(){
      System.out.print("\nDrawing Rectangle.\n");
   }
```

```
    public static void main(String args[]){
        Rectangle r1 = new Rectangle();
        r1.drawRectangle();
        r1.message(); //calling default method of an interface.
        Scanner sc = new Scanner(System.in);
        double length, breadth;
        System.out.print("\nEnter length and breadth of the rectangle: ");
        length = sc.nextDouble();
        breadth = sc.nextDouble();
        double Area = Drawable.area(length,breadth);
        double peri = Drawable.perimeter(length,breadth);
        System.out.print("\nArea : " + Area+" square units."+
            "\nPerimeter: "+peri+ " units");
    }
}
```

static method of Rectangle interface is called here.

What is marker or tagged interface?

An interface which has no member is known as marker or tagged interface, for example Serializable, Cloneable, Remote etc. They are used to provide some essential information to the JVM so that JVM will perform some useful operations.

Java Nested interface

An interface, i.e., declared within another interface or class known as a nested interface. The interface are used to group related interfaces so that they can be easy to maintain. The nested interface must be referred to by the outer interface or class. It can be accessed directly.

Syntax:

```
 interface interfaceName{
     interface nestedInterfaceName{
     …
     }
     …
}
```

There are some points to remember for nested interface by the Java programmer:

- The nested interface must be public if it is declared inside the interface, but it can have any access modifier if declared within the class.
- Nested interfaces are declared static.

Syntax of nested interface which is declared within the class

```
 class className{
      interface nested_interface_name{
     ….
     }
}
```

For example :

```
public interface Showable
{
   /**
    * An example of a method header - replace this comment with your own
    *
```

```
{
  /**
   * An example of a method header - replace this comment with your own
   *
   * @param  y a sample parameter for a method
   * @return   the result produced by sampleMethod
   */
  void show();
  interface Message{
     void msg();
  }
}
```

Compiler

```
public static interface Showable$Message{
     public abstract void msg();
}
```

```
public class NestedInterfaceTest implements Showable.Message
{
   public void msg(){
      System.out.print("\nI am the method of nested interface");
   }
   public static void main(String args[]){
      Showable.Message m = new NestedInterfaceTest(); //upcasting here
      m.msg();
   }
}
```

Example of nested interface inside the class

```
public class AnyClass
{
   interface Message{
      void msg();
   }
}
public class TestNestedInterface2 implements AnyClass.Message
{
   public void msg(){
      System.out.print("\nI am the abstract method of nested interface "+
         "present in AnyClass");
   }
   public static void main(String args[]){
      AnyClass.Message am = new TestNestedInterface2(); //upcasting here
      am.msg();
   }
}
```

Difference between Abstract class and interface

Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated.

But there are many differences between abstract class and interface that are given below.

Abstract class	**Interface**
Abstract class can **have abstract and non-abstract** methods.	Interface can **have only abstract methods.** Since Java 8, it can have **default and static methods.**
Abstract class **doesn't support multiple inheritance.**	Interface supports **multiple inheritance.**
Abstract class **can have final, non-final, static and**	Interface has **only static and final variables.**

non-static variable.	
Abstract class **can provide the implementation of interface.**	Interface **can't provide the implementation of abstract class.**
The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.
An **abstract class** can **extend another Java class and implement multiple interfaces.**	An **interface** can **extend another Java interface only.**
An **abstract class can be extended using keyword "extends".**	An **interface** can be implemented using **keyword "implements"**
A Java **abstract class** can have class members like private protected etc.	Members of Java interface are public by default.
Abstract class achieves partial abstraction(0 to 100%)	Interface achieves fully abstraction(100%)